Kelly Valencia
FORECASTING & BIG DATA
December 10, 2025

Unemployment Forecasting-Time Series & ML

1. The model that yields the best forecast accuracy is the tree regression with the lowest RMSE of 1.38. Ranked from best to worst model.
   a) Tree regression (RMSE = 1.38)
   b) Ridge Regression (RMSE = 1.87)
   c) Lasso (RMSE = 2.12)
   d) Arimax (RMSE = 2.15)
   e) Arima (RMSE = 2.21)

   A possible reason why the tree ended up doing better is because it can pick up cutoff points and interactions in the predictors, reacting differently depending on the state of the economy. While, linear models like Lasso and Ridge assume smooth and constant relationships between predictors and unemployment, which makes them less flexible where relationships can change depending on how the economy is doing. Arima also underperformed, likely because it didn't use the additional predictors aside from past unemployment. Arimax's RMSE slightly improved compared to Arima when the predictors were added, but it was also outperformed by the other models. It still assumes constant effects and it also used many parameters making it a complicated model.

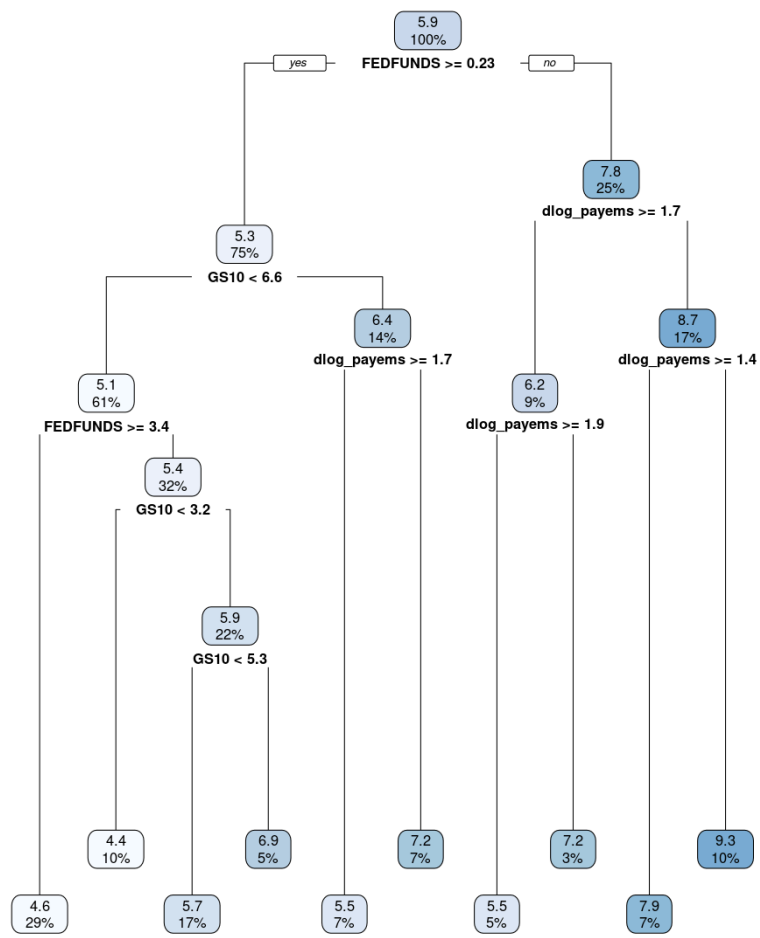
```
> ridge_rmse; ridge_mae
[1] 1.867192
[1] 1.183049
> arimax_rmse; arimax_mae
[1] 2.15482
[1] 1.738189
> arima_rmse; arima_mae
[1] 2.212164
[1] 0.992135
> lasso_rmse; lasso_mae
[1] 2.114974
[1] 1.331113
> rmse_tree; mae_tree
[1] 1.376954
[1] 1.06939
```

2. Arimax's RMSE did improve slightly, dropping from 2.21 to 2.15, when it used predictors compared to plain univariate Arima. This makes sense since adding predictors gives the model extra signals about economic conditions, helping it capture movements that a purely univariate Arima cannot. However, Arimax was still outperformed by the other models. Interestingly, the mae worsened, going from 0.99 to 1.73 when the predictors were added. So, arimax made more mistakes on average than arima (MAE), but it did reduce a few large forecast errors (RMSE). RMSE rewards fixing big errors, while MAE penalizes consistent errors so they can move in opposite directions.

```
> arimax_rmse; arimax_mae
[1] 2.15482
[1] 1.738189
> arima_rmse; arima_mae
[1] 2.212164
[1] 0.992135
```

3. Ridge and LASSO identified essentially the same predictors as most influential when looking at the absolute size of their coefficients despite their values differing. The interest rate variables (FEDFUNDS and GS10) were among the strongest predictors, along with key growth measures such as dlog_pce (consumption growth) and dlog_payems (payroll employment growth). In the Ridge model, dlog_payems had a coefficient of –0.34, dlog_pce was -0.46, FEDFUNDS was -0.49, and GS10 was 0.40. LASSO produced a similar pattern with dlog_payems = -0.30, dlog_pce = -0.57, FEDFUNDS = -0.63, and GS10 = 0.58. For the regression tree, the predictors that primarily decide how the tree splits are the most important. FEDFUNDS is a root split, so that one would be the most important, followed by GS10 and dlog_payems which appear in several splits.

```
> coef(lasso_model)
7 x 1 sparse Matrix of class "dgCMatrix"
                    s0
(Intercept)  7.1437311
dlog_cpi     0.2377866
dlog_indpro  0.2270357
dlog_payems -0.3039779
dlog_pce    -0.5756621
FEDFUNDS    -0.6306703
GS10         0.5777566
> coef(ridge_model)
7 x 1 sparse Matrix of class "dgCMatrix"
                    s0
(Intercept)  7.2526932
dlog_cpi     0.1791521
dlog_indpro  0.1881381
dlog_payems -0.3383357
dlog_pce    -0.4611358
FEDFUNDS    -0.4902874
GS10         0.4024352
```



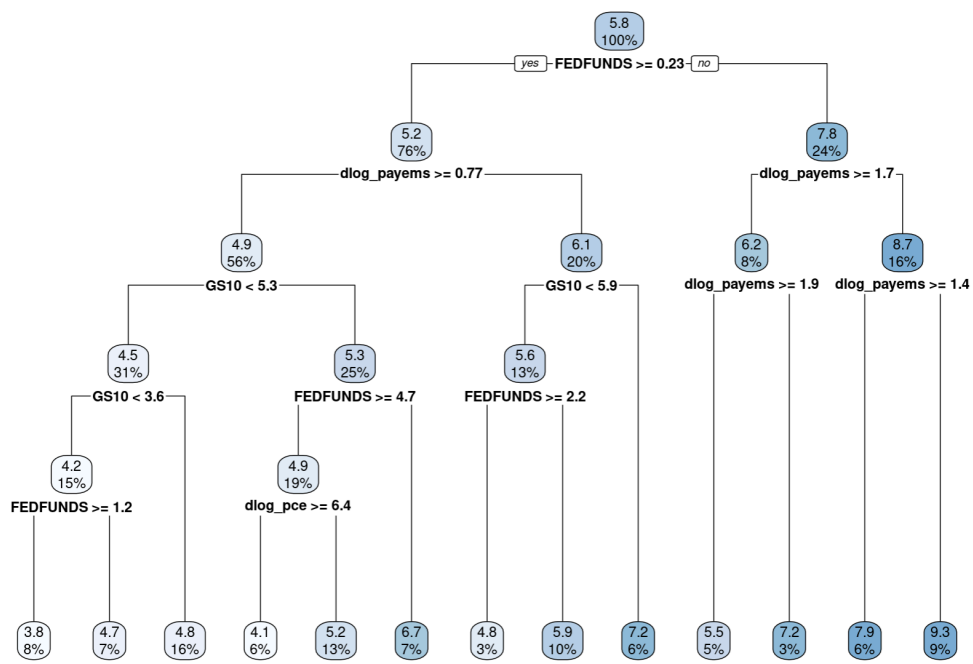

```
> t_pruned$variable.importance
   FEDFUNDS        GS10 dlog_payems    dlog_pce    dlog_cpi dlog_indpro
  528.09582   401.63332   329.49237   168.50791   166.52623    95.25438
```

4. Used code

```
> covid_start <- as.Date("2020-03-01")
> train_data <- data_ts |> filter(date < covid_start)
> test_data  <- data_ts |> filter(date >= covid_start)
```

I chose to use March as the cutoff since the shutdowns began around January-March. Overall, RMSE for the models slightly increased except for Arimax. Tree, Ridge, and Lasso were mostly unchanged, showing only slightly higher RMSE when forecasting the COVID shock. The noticeable difference was Arima, which performed much worse with an RMSE of 2.73. Since it only uses past unemployment information, it was unable to anticipate a shock like covid. Arimax improved slightly, but it was still outperformed by the other models. The regression tree also produced more splits when forecasting the covid period showing the model was attempting to capture its volatile shocks.

```
> ridge_rmse; ridge_mae
[1] 1.895754
[1] 1.109889
> arimax_rmse; arimax_mae
[1] 2.051014
[1] 1.241227
> arima_rmse; arima_mae
[1] 2.737315
[1] 1.851113
> lasso_rmse; lasso_mae
[1] 2.192416
[1] 1.365661
> rmse_tree; mae_tree
[1] 1.481719
[1] 1.149444
```



```
> library(fredr)
> library(dplyr)
> library(lubridate)
> library(tidyr)
> library(ggplot2)
> library(forecast)
> library(glmnet)
> library(rpart)
> library(rpart.plot)
>
> set.seed(123)
> fredr_set_key("25e01a7269f9d0fdd83a00ee95e51aee")
>
> get_fred_series <- function(series_id, from = "1990-01-01") {
+  fredr(series_id = series_id,
+        observation_start = as.Date(from)) |>
+    select(date, value) |>
+    rename(!!series_id := value)
+ }
>
> unrate <- get_fred_series("UNRATE")
>
> fedfunds <- get_fred_series("FEDFUNDS")
> cpi      <- get_fred_series("CPIAUCSL")
> indpro   <- get_fred_series("INDPRO")
> payems   <- get_fred_series("PAYEMS")
> pce      <- get_fred_series("PCE")
> gs10     <- get_fred_series("GS10")
>
> data_raw <- unrate |>
+  left_join(fedfunds, by = "date") |>
+  left_join(cpi,      by = "date") |>
+  left_join(indpro,   by = "date") |>
+  left_join(payems,   by = "date") |>
+  left_join(pce,      by = "date") |>
+  left_join(gs10,     by = "date")
>
> data <- data_raw |>
+  drop_na()
>
> data <- data |>
+  arrange(date) |>
+  mutate(
+    dlog_cpi    = 100 * (log(CPIAUCSL) - dplyr::lag(log(CPIAUCSL), 12)),
+    dlog_indpro = 100 * (log(INDPRO) - dplyr::lag(log(INDPRO), 12)),
+    dlog_payems = 100 * (log(PAYEMS) - dplyr::lag(log(PAYEMS), 12)),
+    dlog_pce    = 100 * (log(PCE) - dplyr::lag(log(PCE), 12))
+  ) |>
+  drop_na()
>
> library(rsample)
>
> data_ts <- data |> arrange(date)
>
> n <- nrow(data_ts)
> train_n <- floor(0.8 * n)
>
> train_data <- data_ts[1:train_n, ]
```

```
> test_data  <- data_ts[(train_n + 1):n, ]
```

```
> #-------------------LASSO---------------
> x <- model.matrix(
+  UNRATE ~ dlog_cpi + dlog_indpro + dlog_payems + dlog_pce + FEDFUNDS + GS10, data
= train_data)[, -1]
> y <- train_data$UNRATE
>
> cv_lasso <- cv.glmnet(x, y, alpha = 1)
>
> best_lambda <- cv_lasso$lambda.min
> best_lambda
[1] 0.00166098
>
> #Fit final LASSO model with lambda.min
> lasso_model <- glmnet(x, y, alpha = 1, lambda = best_lambda)
>
> #Get coefficients
> coef(lasso_model)
7 x 1 sparse Matrix of class "dgCMatrix"
                     s0
(Intercept)  7.1437311
dlog_cpi     0.2377866
dlog_indpro  0.2270357
dlog_payems -0.3039779
dlog_pce    -0.5756621
FEDFUNDS    -0.6306703
GS10         0.5777566
>
> X_test <- model.matrix(UNRATE ~ dlog_cpi + dlog_indpro + dlog_payems + dlog_pce +
FEDFUNDS + GS10, data = test_data)[, -1]
>
> lasso_pred <- predict(lasso_model, newx = X_test)
>
> #Build results for accuracy
> lasso_results <- test_data |> mutate(.pred = as.numeric(lasso_pred))
>
> lasso_rmse <- sqrt(mean((lasso_results$UNRATE - lasso_results$.pred)^2))
> lasso_mae  <- mean(abs(lasso_results$UNRATE - lasso_results$.pred))
>
> lasso_rmse; lasso_mae
[1] 2.114974
[1] 1.331113
```

```
> #---------------ARIMA---------------
> un_ts <- ts(data$UNRATE, start = c(1990, 1), frequency = 12)
> fit_arima <- auto.arima(un_ts)
>
> un_train <- ts(train_data$UNRATE, frequency = 12)
> un_test  <- test_data$UNRATE
>
> # Fit ARIMA on train only
> fit_arima <- auto.arima(un_train)
>
> # Forecast ahead for exactly the length of the test set
> fc_arima <- forecast(fit_arima, h = length(un_test))
>
> # Errors
> arima_rmse <- sqrt(mean((fc_arima$mean - un_test)^2))
> arima_mae  <- mean(abs(fc_arima$mean - un_test))
>
```

```
> coef(fit_arima)
       ar1        ar2        ar3        ma1        ma2        ma3
 0.1966985 -0.1336069  0.8163447 -0.2508707  0.2568911 -0.6700982
       ma4        ma5       sar1       sar2       sma1
 0.1474589  0.1207289  0.4010859 -0.1419445 -0.6757651
> arima_rmse;arima_mae
[1] 2.212164
[1] 0.992135
> fit_arima
Series: un_train
ARIMA(3,1,5)(2,0,1)[12]

Coefficients:
         ar1      ar2     ar3      ma1     ma2      ma3     ma4
      0.1967  -0.1336  0.8163  -0.2509  0.2569  -0.6701  0.1475
s.e.  0.1418   0.2598  0.3379   0.1528  0.2517   0.3447  0.1224
         ma5    sar1     sar2     sma1
      0.1207  0.4011  -0.1419  -0.6758
s.e.  0.0640  0.1052   0.0678   0.0897

sigma^2 = 0.01812:  log likelihood = 198.19
AIC=-372.39   AICc=-371.41   BIC=-326.73
```

```
> #---------------ARIMAX---------------
> y_train <- ts(train_data$UNRATE, frequency = 12)
> y_test  <- test_data$UNRATE
>
> # Predictors: dlogs + level interest rates
> x_train <- as.matrix(train_data[, c(
+  "dlog_cpi", "dlog_indpro", "dlog_payems", "dlog_pce",
+  "FEDFUNDS", "GS10"
+ )])
>
> x_test <- as.matrix(test_data[, c(
+  "dlog_cpi", "dlog_indpro", "dlog_payems", "dlog_pce",
+  "FEDFUNDS", "GS10"
+ )])
>
> # Fit ARIMAX
> fit_arimax <- auto.arima(y_train, xreg = x_train)
> summary(fit_arimax)
Series: y_train
Regression with ARIMA(2,0,5)(2,0,1)[12] errors

Coefficients:
         ar1      ar2      ma1     ma2      ma3     ma4     ma5
      1.9750  -0.9765  -1.1574  0.2232  -0.0087  0.0171  0.0686
s.e.  0.0142   0.0141   0.0599  0.0852   0.0888  0.0822  0.0617
        sar1     sar2     sma1  intercept  dlog_cpi  dlog_indpro
      0.3715  -0.1221  -0.6532     6.1339   -0.0163      -0.0226
s.e.  0.1105   0.0689   0.0983     0.3502    0.0203       0.0105
      dlog_payems  dlog_pce  FEDFUNDS     GS10
          -0.1899    0.0141   -0.0219  -0.0020
s.e.       0.0552    0.0155    0.0410   0.0259

sigma^2 = 0.01662:  log likelihood = 213.96
AIC=-391.92   AICc=-389.74   BIC=-323.37

Training set error measures:
                       ME      RMSE       MAE         MPE
Training set 0.0002319969 0.1255753 0.1013064 -0.07335241
```

```
                  MAPE      MASE          ACF1
Training set 1.791793 0.1438273 -0.0009519091
>
> # Forecast over test window
> fc_arimax <- forecast(fit_arimax, xreg = x_test, h = length(y_test))
>
> arimax_pred <- as.numeric(fc_arimax$mean)
>
> arimax_rmse <- sqrt(mean((arimax_pred - y_test)^2))
> arimax_mae  <- mean(abs(arimax_pred - y_test))
>
> coef(fit_arimax)
         ar1          ar2          ma1          ma2          ma3
 1.974964906 -0.976507303 -1.157353673  0.223181224 -0.008735076
         ma4          ma5         sar1         sar2         sma1
 0.017125354  0.068624465  0.371531081 -0.122066943 -0.653224530
   intercept     dlog_cpi  dlog_indpro  dlog_payems     dlog_pce
 6.133921018 -0.016276924 -0.022641739 -0.189889660  0.014077793
    FEDFUNDS         GS10
-0.021900815 -0.002027755
> arimax_rmse;arimax_mae
[1] 2.15482
[1] 1.738189
> fit_arimax
Series: y_train
Regression with ARIMA(2,0,5)(2,0,1)[12] errors

Coefficients:
         ar1      ar2      ma1     ma2      ma3     ma4     ma5
      1.9750  -0.9765  -1.1574  0.2232  -0.0087  0.0171  0.0686
s.e.  0.0142   0.0141   0.0599  0.0852   0.0888  0.0822  0.0617
        sar1     sar2     sma1  intercept  dlog_cpi  dlog_indpro
      0.3715  -0.1221  -0.6532     6.1339   -0.0163      -0.0226
s.e.  0.1105   0.0689   0.0983     0.3502    0.0203       0.0105
      dlog_payems  dlog_pce  FEDFUNDS     GS10
          -0.1899    0.0141   -0.0219  -0.0020
s.e.       0.0552    0.0155    0.0410   0.0259

sigma^2 = 0.01662:  log likelihood = 213.96
AIC=-391.92   AICc=-389.74   BIC=-323.37
```

```
> #---------------RIDGE---------------
> # 1) Design matrix and response (TRAIN)
> x_train <- model.matrix(
+  UNRATE ~ dlog_cpi + dlog_indpro + dlog_payems + dlog_pce +
+    FEDFUNDS + GS10,
+  data = train_data
+ )[, -1]
>
> y_train <- train_data$UNRATE
>
> # 2) Cross-validated Ridge
> ridge_cv <- cv.glmnet(x_train, y_train, alpha = 0)
>
> best_lambda_ridge <- ridge_cv$lambda.min
> best_lambda_ridge
[1] 0.08461262
>
> # 3) Final Ridge model
> ridge_model <- glmnet(
+  x_train, y_train,
```

```
+  alpha = 0,
+  lambda = best_lambda_ridge
+ )
>
> # 4) Test design matrix
> x_test <- model.matrix(
+  UNRATE ~ dlog_cpi + dlog_indpro + dlog_payems + dlog_pce +
+    FEDFUNDS + GS10,
+  data = test_data
+ )[, -1]
>
> # 5) Predictions + errors
> ridge_pred <- predict(ridge_model, newx = x_test)
> ridge_pred <- as.numeric(ridge_pred)
>
> ridge_rmse <- sqrt(mean((ridge_pred - test_data$UNRATE)^2))
> ridge_mae  <- mean(abs(ridge_pred - test_data$UNRATE))
>
> coef(ridge_model)
7 x 1 sparse Matrix of class "dgCMatrix"
                    s0
(Intercept)  7.2526932
dlog_cpi     0.1791521
dlog_indpro  0.1881381
dlog_payems -0.3383357
dlog_pce    -0.4611358
FEDFUNDS    -0.4902874
GS10         0.4024352
> ridge_rmse; ridge_mae
[1] 1.867192
[1] 1.183049
>
> #----
> ridge_rmse; ridge_mae
[1] 1.867192
[1] 1.183049
> arimax_rmse; arimax_mae
[1] 2.15482
[1] 1.738189
> arima_rmse; arima_mae
[1] 2.212164
[1] 0.992135
> lasso_rmse; lasso_mae
[1] 2.114974
[1] 1.331113
> rmse_tree; mae_tree
[1] 1.376954
[1] 1.06939
```

```
> #COVID SPLIT
> #train_data <- data_ts[1:train_n, ]
> #test_data  <- data_ts[(train_n + 1):n, ]
>
> covid_start <- as.Date("2020-03-01")
> train_data <- data_ts |> filter(date < covid_start)
> test_data  <- data_ts |> filter(date >= covid_start)
> ridge_rmse; ridge_mae
[1] 1.895754
```

```
[1] 1.109889
> arimax_rmse; arimax_mae
[1] 2.051014
[1] 1.241227
> arima_rmse; arima_mae
[1] 2.737315
[1] 1.851113
> lasso_rmse; lasso_mae
[1] 2.192416
[1] 1.365661
> rmse_tree; mae_tree
[1] 1.481719
[1] 1.149444
```